

# MoE 训练论文翻译·第三部分：突破三堵墙

本节是论文最核心的工程优化部分，重点介绍内存、通信、计算效率三堵墙的系统化解决方案。

## 4. 扩展 MoE：突破内存、通信与计算效率墙

三堵墙为何相互作用：这些挑战并非独立，而是紧密耦合的系统。举例说明：一个团队训练 1000B MoE 模型时，首先遇到内存墙 → 启用激活重计算 → 内存问题解决后，all-to-all 通信成为主要瓶颈 → 实现通信-计算重叠 → 引入低精度训练扩大批大小 → 量化内核碎片化执行流，主机侧启动成为瓶颈 → 部署 CUDA Graphs → 但 CUDA Graphs 需要静态张量形状，与 dropless 路由冲突。每个解决方案都在系统中产生涟漪，需要同时关注三堵墙。

### 4.1 突破内存墙

内存是首要硬约束：若参数、优化器状态和激活的组合占用超过 GPU 容量，训练根本无法进行。

#### 4.1.1 内存解剖：内存消耗的来源

以 DeepSeek-V3（BF16 精度，PP4×VPP4×EP64，256 GPU）为例：

| 组件        | 每 GPU 内存 | 优化技术          |
|-----------|----------|---------------|
| 权重和梯度     | 36.4 GB  | PP/EP/TP 分片   |
| 主权重和优化器状态 | 32.1 GB  | 分布式优化器，BF16 矩 |
| 激活        | 131.0 GB | 低精度、重计算、卸载    |
| 总计        | 199.5 GB | ——            |

⚠ 关键洞察：激活主导了内存消耗，超过权重、梯度和优化器状态的总和。激活内存优化是首要优先级。

四种互补策略：

- 降低存储精度：FP8/FP4 代替 BF16，内存高效排列消除冗余中间张量
- 用计算换存储：激活重计算在前向传递中丢弃中间结果，反向时重新生成
- 卸载到主机内存：前向时转移到 CPU，反向时取回
- 跨设备分发：FSDP 将参数、梯度和优化器状态分片到数据并行 ranks

4.1.2 内存高效排列：零开销激活减少

最理想的内存优化是无计算开销的。内存高效排列通过简单的代数重排消除冗余中间张量，实现零额外开销。

原理：考虑 token  $x$  被路由到其 top- $k$  专家。标准公式（路由权重在专家计算后应用）：

$$y = \sum_{i \in T(x)} p_i \cdot W_2^{(i)} \cdot \phi(W_1^{(i)} \cdot x) \quad \text{—— 公式(1)}$$

内存高效排列（将  $p_i$  吸收到激活中，在第二个线性层之前应用）：

$$y = \sum_{i \in T(x)} W_2^{(i)} \cdot (p_i \cdot \phi(W_1^{(i)} \cdot x)) \quad \text{—— 公式(2)}$$

当专家没有偏置项时， $W_2^{(i)}$  是纯线性映射，标量乘法可交换，两个公式数学等价。

节省机制：在标准公式中，计算  $\partial L / \partial p_i$  需要在整个反向传递过程中保留每个专家输出  $E_i(x)$ 。在内存高效公式中， $p_i$  直接乘以  $\phi(z_i)$ （预激活输入的激活函数），反向内核可从  $z_i$  即时重计算  $\phi(z_i)$ ，而  $z_i$  本身无论如何都需要保存用于 SwiGLU 的反向传递。因此不引入额外缓冲区，净减少峰值内存，零计算开销。

效果：对 DeepSeek-V3，内存高效排列每 GPU 节省约 **26.3 GB** 激活内存。

---

4.1.3 低精度训练：FP8/FP4 激活内存减少

将前向传递中保存用于梯度计算的激活（主要是每个线性层的输入）从 BF16 改为 FP8/FP4 存储，每个张量内存减少 50%/75%。

适用范围：注意力投影（Q/K/V/输出）和 MLP 层（包括 MoE 专家 MLP）的线性层输入。不适用的激活（注意力分数、归一化中间值、路由张量）要么需要更高精度保证数值稳定性，要么不跨前向-反向边界存储。

效果：对 DeepSeek-V3，FP8 训练每 GPU 减少约 **16 GB** 激活内存（约 12% 的 131 GB 激活预算）。

---

4.1.4 重计算：用计算换内存

激活重计算（激活检查点）是成熟技术，但全层重计算可能增加约 33% 计算开销，对 MoE 层代价更高（重计算专家计算还会重触发 EP all-to-all 通信）。Megatron-Core 引入细粒度重计算，只针对内存最密集但计算最廉价的操作。

两种组合技术：

细粒度重计算（Granular Recomputation）用户精确指定在反向传递中重计算哪些计算。例如只重计算专家 MLP 中的激活函数、LayerNorm 模块或 MLA 中的 up-projection。通过重计算单个操作或子模块，可以以小于 5% 的额外计算开销实现显著的内存节省。

**输出丢弃重计算（Output-Discarding Recomputation）** 传统激活检查点将检查点模块的输出传递到后续层，并存储这些输出以备反向传播。但由于这些输出在反向传递中会被重计算，存储是冗余的。Megatron-Core 在检查点模块的输出被后续层消耗后立即释放，反向时从重计算结果恢复。这确保内存不会被可以廉价恢复的激活不必要地占用，在不损害梯度正确性的前提下减少内存占用。

量化效果（DeepSeek-V3 配置）：

| 重计算目标             | 每 GPU 节省内存 |
|-------------------|------------|
| MLA Up-Projection | 30.4 GB    |
| 激活函数（SwiGLU）      | 3.8 GB     |
| LayerNorm         | 8.2 GB     |
| 总计                | 42.4 GB    |

4.1.5 细粒度激活卸载

当 GPU 内存在精度和重计算优化后仍然不足时，将激活卸载到 CPU 内存提供额外容量。与以计算换内存的重计算不同，卸载以 PCIe 带宽换内存。

动机：细粒度 MoE 模型有极端的参数膨胀（DeepSeek-V3 每 token 只激活 37B，18× 比例；Kimi-K2 达到 31× 比例）。**EP** 和 **PP** 不减少激活内存（这些策略减少参数内存，不是激活内存），使卸载尤为关键。

核心思想：重叠与预取

GPU 的复制引擎和计算引擎独立运行。当模块计算时间超过激活传输时间时，D2H（设备到主机）复制可以与后续计算并发运行，零额外开销。

前向传递：模块计算完成后立即卸载输入激活到 CPU，通过专用 D2H 流与下一模块的计算并行运行。例外：最后一层激活不卸载（因为反向立即需要，没有计算可隐藏传输延迟）。

反向传递：采用层交错重载模式（**Layer-Staggered Reload**）：在计算当前层梯度时，从下一层预取同一模块的激活。重载在每个模块的反向完成后发生，同一时刻每种模块类型只有一个激活驻留在 GPU 内存中，避免内存翻倍。

相比全重计算的峰值内存优势：

- 全重计算：GPU 峰值内存 =  $L \times \text{layer\_input} + 1 \times \text{layer\_intermediate}$ （与层数成正比！）
- 卸载：每个输入在使用前刚好重载，立即释放。GPU 峰值内存减少至  $1 \times \text{layer\_input} + 1 \times \text{layer\_intermediate}$ （独立于模型深度！）

对 60+ 层的深度模型，这是全重计算无法实现的根本内存优势。

关键技术特性：

- 模块级粒度：用户通过 `--offload-modules` 指定卸载哪些模块，支持混合策略（轻量模块用重计算，昂贵模块用卸载）
- 异步传输：专用 D2H/H2D CUDA 流与计算并行运行，只在必要时通过 CUDA 事件协调
- 重计算集成：与细粒度重计算结合（对 `moe_act`），两种策略都适用：重计算激活同时卸载其输入）
- **CUDA Graphs 兼容**：使用外部事件而非流同步，允许卸载模块被图外计算隐藏

量化效果：

| 模型和配置                               | 基线内存   | +卸载内存  | 内存变化   | 吞吐量变化         |
|-------------------------------------|--------|--------|--------|---------------|
| DeepSeek-V3 完整 TP1PP8EP32VPP4 MXFP8 | 169 GB | 151 GB | -10.7% | -1.6%         |
| Qwen3-235B TP2→TP1 + EP16→EP64      | 172 GB | 175 GB | +1.7%  | <b>+15.0%</b> |

Qwen3-235B 的卸载降低了所需的 TP 度，反而提升了 15% 的吞吐量。

4.1.6 权重和优化器优化：低精度存储与卸载

精度感知优化器：Adam 优化器每个参数维护两个状态张量（`exp_avg` 和 `exp_avg_sq`），传统 FP32 存储每参数消耗 8 字节。关键洞察：优化器状态可以容忍更低的存储精度，只要实际更新计算保持在更高精度。

精度感知优化器将存储精度与计算精度解耦：第一/二矩可以存储在 BF16（每个 2 字节）甚至 FP8（每个 1 字节），在优化器步骤中动态转换为 FP32 进行计算。典型配置：主参数和梯度保持 FP32，矩估计存储在 BF16——实现约 **50%** 的优化器状态内存减少（从 32.1 GB 预算节省约 10-12 GB），对训练动态几乎无影响。

状态卸载：将优化器状态（`exp_avg`, `exp_avg_sq`）和主权重在 `optimizer.step()` 后迁移到 CPU，下一步前重载。前向和反向传递期间优化器状态占用 GPU 内存但保持非活动，卸载回收这部分内存供其他操作使用（保持优化器步骤在 GPU 上执行）。

对 DeepSeek-V3，状态卸载节省 **15-20 GB** GPU 内存，每次迭代额外开销仅 0.1-0.2 秒。

4.1.7 FSDP for MoE

FSDP（完全分片数据并行）将模型参数、梯度和优化器状态跨数据并行 ranks 分片，每个 GPU 只持有本地分片。专家参数常常主导 MoE 模型内存，FSDP 是 EP 的自然补充。

为什么 FSDP 适合 MoE：

- 内存减少和通信减少：EP 为每个 GPU 分配专家子集，FSDP 在专家数据并行（EDP）组内进一步分片这些本地专家，per-GPU 内存和集合通信体积都随 EDP 大小扩展
- 无 PP 灵活性：FSDP+EP 避免了 PP 的工程痛点（不均匀 VPP 阶段均衡、DeepSeek 式模型的输出/MTP 层放置等）

**FSDP+EP 双 DeviceMesh 设计：**核心挑战是密集层和专家层需要不同的分片范围。Megatron-FSDP 通过双 DeviceMesh 架构解决：

- 主 DeviceMesh 管理密集模块（注意力、归一化）的 DP 分片
- 辅助专家 DeviceMesh 管理 EP 模块，FSDP 作用域在 EDP 维度

每个 transformer 层自动将子模块路由到适当的 mesh。注意力/归一化使用主 mesh，MoE 专家 FFN 层使用专家 DeviceMesh。

零复制通信：

1. 非均匀分片：标准 FSDP2 独立分片每个参数，Megatron-FSDP 将模块内所有参数展平并拼接，然后进行非均匀分片——分片边界与通信缓冲区布局对齐，集合通信可直接从展平存储读取，无冗余复制。在 Llama3 405B 训练中减少约 10% 通信开销。
2. 持久双缓冲区 + NCCL 用户缓冲区注册（UBR）：基线 FSDP 频繁分配释放通信缓冲区，NCCL 在内部缓冲区间复制数据。Megatron-FSDP 在训练开始时预分配两个持久缓冲区并在它们之间循环，同时通过 UBR 注册给 NCCL，实现真正零复制通信。在 NVLink 系统上，通信内核的 SM 占用从 8-32 个 SM 降至 1-4 个 SM。

4.1.8 内存优化总结

| 技术         | 目标内存   | 权衡           |
|------------|--------|--------------|
| 低精度训练      | 激活     | 数值精度，CPU 开销  |
| 内存高效排列     | 激活     | 无额外开销        |
| 细粒度重计算     | 激活     | 计算开销         |
| 细粒度卸载      | 激活     | CPU 开销，非重叠复制 |
| 精度感知优化器    | 优化器状态  | 数值精度         |
| FSDP（含 EP） | 参数+优化器 | 通信开销         |

这些优化相互补充：高效排列零开销消除冗余存储；FP8/FP4 激活以最小质量影响降低精度；细粒度重计算为特定模块提供有利的计算-内存权衡；激活卸载在其他技术不足时提供额外余量；低精度存储和状态卸载减少权重和优化器内存；FSDP 支持超越单设备容量的扩展。

---

## 4.2 突破通信墙

量化问题的严重性：优化前，EP all-to-all 通信通常消耗 **20-60%** 的训练时间（EP 在 NVLink 域内约 20%；EP 跨节点则上升到 40-60%）。

**EP 通信模式**：每个 MoE 层需要两次集合通信——分发（dispatch）将 token 发送到分配的专家，合并（combine）将结果返回。对于有 L 个 MoE 层的模型，每次前向传递涉及 2L 次分发/合并操作，每次传输  $O(Bh)$  数据。

DeepSeek-V3 规模：58 个 MoE 层  $\times 2 = 116$  次操作/前向传递，反向再翻倍。在 50 GB/s 节点间带宽（InfiniBand NDR），单次 200 MB 负载的分发需要数毫秒，每次迭代累积达数百或数千毫秒。

两大优化策略：

1. 最大化带宽利用率：优化分发器（DeepEP、HybridEP）接近峰值带宽
2. 在计算背后隐藏延迟：通信-计算重叠

---

### 4.2.1 通信解剖：专家并行模式

标准 EP 中，all-to-all 有固有限制：在分发前需要排列阶段（每个 token 复制 top-k 次，产生冗余流量），在某些设置中还有主机开销。

### 4.2.2 DeepEP 和 HybridEP：最大化 EP 带宽

为解决这些问题，Megatron-Core 提供两种基于 token 的分发后端。基于 **token** 的分发消除排列步骤，避免发送冗余 **token**，减少总通信体积，提高有效带宽。

**HybridEP**（NVIDIA 开发）：遵循与 DeepEP 相同的基于 token 原则，利用 TMA 和 IBGDA 等硬件原语，目标是以更低 SM 使用率实现相当或更高带宽，包括支持多节点 NVLink（MNNVL）部署。

**HybridEP 分发内核设计**：

- 根据路由信息从全局内存读取数据到共享内存，通过 FIFO 队列将 token 写入目的地
- 节点间情况：不直接发送重复负载，而是先使用 RDMA warp 组在跨节点相同本地索引的 GPU 间交换数据，然后在每个节点内转发，减少跨节点流量并允许节点间和节点内传输重叠

**HybridEP 合并内核设计**：

- 将规约融入通信内核，通过 FIFO 队列读取数据、执行规约，直接写入目标位置（标准 all-to-all 分发只通信，需要单独的反排列步骤）

性能对比（EP 扩展性能，隐藏大小 7168，序列长度 4096，256 个专家，单位：微秒）：

| EP 规模 | GB200 HybridEP | GB200 all-to-all | H100 HybridEP | H100 all-to-all |
|-------|----------------|------------------|---------------|-----------------|
| 8     | 391            | 735              | 661           | 1265            |
| 16    | 578            | 743              | 1485          | 5774            |
| 32    | 612            | 769              | 3064          | 8059            |
| 64    | 675            | 930              | 4626          | 9164            |

HybridEP 在所有测试设置中始终优于 all-to-all，节点间场景下提升更大。

### 4.2.3 EP 通信重叠：隐藏 EP 通信延迟

优化分发器改善带宽利用率，但根本问题依然存在：all-to-all 仍在端到端关键路径上。对 DeepSeek-V3（EP64），EP all-to-all 通信仍可占迭代时间的 30-40%。

关键观察：当重叠窗口内存在足够独立工作时，all-to-all 延迟可以隐藏在计算背后。对 DeepSeek-V3/Kimi-K2 等细粒度 MoE，这很有挑战性——跨节点 EP 通信可消耗约一半层时间，相邻算子（如 GEMM）往往太短无法完全隐藏。

解决方案：专用 **1F1B all-to-all** 重叠方案

Megatron-Core 使用专用方案，合并相邻微批次的前向和反向传递，在 CUDA 流间交错计算和 all-to-all 内核：一个微批次的反向 **all-to-all** 与另一个微批次的前向注意力/**MLP** 重叠。概念上是在标准 1F1B 之上构建的类 DualPipe 双向调度，同时保持 Megatron-Core 兼容性。

两种合并模式：

模式1：合并 **FWD-FWD / BWD-BWD** 两个微批次的同类传递并行运行并重叠 all-to-all。权衡：2× 峰值激活内存；由于前向计算约为反向的一半，重叠 all-to-all 机会较少。

模式2：合并 **FWD-BWD**（首选，**DualPipe** 等效） 将一个微批次的前向与另一个的反向合并（例如微批次1的前向与微批次0的反向）。优势：无额外内存开销（前向激活被后向直接复用）；限制：首个前向和最后反向仍在关键路径上，它们的 all-to-all 无法隐藏。

最大化 **all-to-all** 隐藏的两大优化：

1. 流分离（**Stream Separation**） 工作负载分为两个 CUDA 流：

- 计算流：运行前向/反向计算（注意力、专家 MLP）
- 通信流：运行 all-to-all 通信（EP 的 token 分发/合并）

通过在流间交替任务，all-to-all 与计算并行——最大化减少空闲周期。

2. **W/D** 分割（权重梯度/数据梯度分割） 一个关键依赖阻止重叠：反向分发（B/dispatch）需要反向 MLP（B/mlp）的输出。将反向 MLP 分为：

- **W/mlp**: 权重梯度计算（独立于 B/dispatch）
- **D/mlp**: 数据梯度计算（输入到 B/dispatch）

分割减少计算流空闲时间：当 F/mlp 单独太短时，W/mlp 可以与 F/mlp 重叠来隐藏 B/dispatch。

综合效果：

- 基线：all-to-all 占迭代时间 30-40%（使用 DeepEP 后）
- 1F1B FWD-BWD + W/D 分割后：EP 通信分享降至 < 5%，实现 93% 的重叠率

限制因素：

- 重叠批次比例：更多微批次提高重叠批次比例
  - **all-to-all** 占比：EP 重叠收益在 all-to-all 主导时更大，尤其跨节点 EP 场景
  - **SM** 划分开销：为 all-to-all 保留 SM 会降低 GEMM 效率。DeepEP 每 GPU 使用 20 个 SM，引入约 20% GEMM 效率开销
- 

#### 4.2.4 通信优化总结

这些通信墙优化协同工作：

- DeepEP/HybridEP 最大化带宽利用率
- FWD-BWD 重叠在计算背后隐藏延迟
- 交错 PP 跨流水线阶段扩展重叠机会
- 灵活 VPP 支持最优负载均衡

合计：将 all-to-all 对训练时间的贡献从 60% 降至 10% 以下。

---

### 4.3 突破计算效率墙

即使内存充足且通信已隐藏，GPU 资源仍可能因内核低效或主机无法足够快地调度工作而处于闲置状态。

#### 4.3.1 计算解剖：低效来源

内核效率问题：细粒度 MoE 架构产生利用率不足 GPU 资源的工作负载。DeepSeek-V3 的 256 个小专家产生 M 维度约 128 个 token/专家的 GEMM，远低于峰值 Tensor Core 利用率所需的数千。

主机开销问题：大量小操作导致内核启动开销，CPU 无法足够快地调度工作以保持 GPU 饱和。三个因素加剧这一问题：

- 细粒度专家：许多独立 GEMM，每个需要单独内核启动



- 低精度训练：额外量化内核增加启动开销
- Droptless 路由：动态 token 数量需要主机-设备同步确定张量形状，CPU 成为关键路径

结果：主机绑定（**Host-Boundedness**）——GPU 内核之间相隔数微秒的主机侧延迟，在性能分析追踪中表现为 GPU 在等待主机工作时的空隙。

---

#### 4.3.2 分组 GEMM 与算子融合：提高内核效率

分组 GEMM（**Grouped GEMM**）：专家计算本质上是一系列独立 GEMM。分组 GEMM 通过重叠内核的波尾效应（Wave Tail Effect）比分离的顺序 GEMM 表现更好。

Megatron-Core 提供两种当前可用实现和两种在研实现：

当前实现：

1. 多流 **cuBLASLt GEMM**：将独立 cuBLASLt GEMM 启动到多个 CUDA 流，允许它们相互重叠。支持多种精度（BF16、per-tensor FP8、blockwise FP8、MXFP8、NVFP4）
2. **CUTLASS Grouped GEMM**：将独立 GEMM 融入单个内核，当 GEMM 数量大时性能更好。当前 TE 中仅支持 Hopper 上的 BF16

在研实现：3. **cuBLASLt Grouped GEMM (CUBLASLT\_BATCH\_MODE\_GROUPED)**：在设备上假设形状信息并在单个内核中执行计算，解锁了对专家部分应用 CUDA Graphs 的可能，覆盖所有精度 4. **cuteDSL Grouped GEMM with Fusions**：通过 cuteDSL 将激活函数、量化（用于下一层）和缩放因子混洗融入 GEMM，针对 Blackwell 上的 MXFP8 和 NVFP4 优化，单个内核整合专家计算、激活和量化，显著减少内核数量，与 CUDA Graphs 兼容

---

#### 4.3.3 排列融合（Permutation Fusion）

分组 GEMM 要求分配给同一专家的 token 连续存储（即排列后的 token）。用原生 PyTorch 实现高效排列很有挑战性，往往启动许多小内核并产生额外 CPU 开销。

排列融合流水线分三个阶段：

1. 预处理：生成偏移映射（Row ID Map），指示每个 token 在输入/输出缓冲区中的偏移。预处理内核只在前向传递的排列前调用一次，其他组件复用结果
  2. 排列（**Permute**）：根据偏移映射将 token 从输入缓冲区移到输出缓冲区。在内存高效排列中，概率也需要排列，结果直接进入专家的激活部分
  3. 反排列（**Unpermute**）：排列的逆操作。token 在排列时被复制多次发送到不同目的地，合并阶段需要将这些 token 求和（内存高效排列下直接相加；否则用概率作为权重）。所有累加以 FP32 精度执行
-

#### 4.3.4 路由器和辅助损失融合

路由器和辅助损失计算是另一个产生大量小内核和 CPU 开销的领域。

路由器中的 GEMM 和通信难以融合，Megatron-Core 将其余操作分解为三个部分，每个部分融合为单个内核：

1. 评分计算（含 top-k 和 softmax/sigmoid）：包含不同模型的各种分支，支持不同 top-k 算法（朴素 top-k、组 top-k）、评分函数（sigmoid、softmax）及其组合
  2. 辅助损失的评分计算：第一部分的子集，在评分函数和朴素 top-k 计算后生成辅助损失计算的输入
  3. MoE 辅助损失计算：融入单个内核
- 

#### 4.3.5 低精度训练：低精度加速

细粒度 MoE 工作负载中小的专家 GEMM 通常受内存带宽限制，限制 Tensor Core 利用率。低精度训练通过减少数据移动和使用更快的 FP8/FP4 Tensor Core 操作来解决这一问题。

收益：FP8/FP4 GEMM 在线性层计算中最大化 Tensor Core 利用率。结合激活内存节省，FP8 训练在不同硬件平台上实现大规模 MoE 训练约 **10-25%** 的端到端性能提升，FP4 提升更多。

---

#### 4.3.6 CUDA Graphs：消除主机开销

内核融合减少内核启动次数；CUDA Graphs 消除这些启动的每次迭代成本。

主机开销的来源：

1. Python 执行：解释器开销、C FFI、垃圾收集
2. 框架开销：即使 `torch.empty` 等简单操作也增加微秒级主机侧成本
3. 内核启动开销：每次内核启动产生数微秒 CPU 侧成本

三种趋势放大了这一问题：

- 更快的 GPU：内核执行加速后，重叠 CPU 工作的时间越来越少，CPU 开销日益明显
- MoE 复杂性：MoE 为 FFN 层增加了路由器、分发、分组 GEMM 和合并操作，产生大量小内核
- 低精度训练：额外的量化内核，在细粒度 MoE 中与专家数量成比例增加

**CUDA Graphs** 的工作原理：在初始迭代中将 GPU 内核序列捕获到可重放图中；后续迭代发出单个 Graph Launch 调用，很大程度上绕过逐操作的 Python/框架开销和逐内核启动开销。

**CUDA Graphs** 的关键挑战：要求所有捕获操作具有静态、预先确定的形状。动态形状或控制流导致图捕获失败——这与 MoE 形成根本矛盾，因为每个专家的 token 数量动态变化。

## 两种 CUDA Graphs 模式：

**全 CUDA Graphs：**将整个前向-反向传递捕获为单个 CUDA Graph。只支持使用 **token** 丢弃和填充的密集或 **MoE** 模型（每个专家有固定接收容量，形状静态）。不适用于 DeepSeek-V3 等 dropless MoE（专家 GEMM 形状逐迭代动态变化）。

**层级 (Partial) CUDA Graphs：**只捕获每层中静态组件：

静态组件（可以图化）：

- 注意力层（处理固定数量 token）
- 路由器计算（固定输入/输出形状）
- EP 预处理（静态排列元数据）
- 共享专家（处理所有 token，固定形状）
- 密集 transformer 块中的 MLP 层

动态组件（不能图化）：

- Token 分发（固定形状到动态形状）
- 专家 GEMM 操作（基于 token 分配的动态 M 维度）
- Token 合并（动态形状到固定形状）

通过以 `"attn+moe_router+moe_preprocess"` 范围启用，一个图捕获每层分发操作前的所有静态组件，实现显著性能提升——即使没有捕获整个模型。

## CUDA Graphs 的内存优化：

- 减少图数量：无 PP 时，微批次可共享同一个图（只需  $L \times 2$  个图）；有 PP 时，每个微批次需要专用图（ $L \times M \times 2$  个图）。引入 `is_first_microbatch` GPU 标志控制共享图内的微批次特定行为
- 内存池共享：所有图可在按执行顺序捕获时共享一个池
- 缓冲区复用：静态输入/输出缓冲区可根据 PP 执行顺序跨图复用

效果：在 DeepSeek-V3 GB200 训练上实现 **10%** 端到端加速，额外内存约 7 GB。

---

### 4.3.7 Dropless MoE 的全 CUDA Graphs 覆盖

剩余问题：在 dropless MoE 中，每个 EP rank 接收的 token 数量根据路由决策动态变化。路由器在 GPU 上生成形状信息，但主机需要它来启动后续操作——这需要设备到主机的复制和同步，将 CPU 置于关键路径上，阻止 CUDA Graphs 捕获动态部分。

两大核心挑战：

1. 不知道实际问题大小的内核启动：主机必须在设备查询每专家 token 数量后才能启动分组 GEMM 或通信内核

2. 不知道实际大小时的内存分配：按最坏情况分配浪费内存；实际大小分配需要同步

三种互补技术实现全覆盖：

### 设备发起的 GPU 内核 (Device-Initiated Kernels)

要消除主机-设备同步，内核必须是设备发起的，要求：

1. GPU 内核从 GPU 内存读取形状信息并用其指导计算
  2. 将实际工作量（运行时才知道）与静态启动配置解耦
  3. 跳过不必要计算（如对填充数据的操作）
- 设备发起分组 **GEMM**：自 CUDA 13.1 起，cuBLASLt Grouped GEMM 支持将矩阵形状作为设备数组传入，内置启发式选择最优内核配置
  - **HybridEP** 同步无关分发：通过估计上界并传递给 HybridEP，分发器可以按上界预分配输出缓冲区，消除所有同步

### ECHO：热专家弹性克隆 (Elastic Cloning for Hot Experts)

**ECHO** 解决两个问题：热专家导致的计算瓶颈（部分 EP ranks 成为计算瓶颈，其他 ranks 等待同步）和最坏情况缓冲区预配置浪费（高负载方差意味着最坏情况比实际使用浪费大量内存）。

工作原理：

- 前向传递：ECHO 规划器识别热专家，生成热专家映射（哪些专家克隆到哪些空闲槽）和更新的路由映射（将溢出 token 重定向到克隆）。专家分发通过 HybridEP 将热专家权重复制到空闲槽；token 分发将 token 路由到原始和克隆专家；所有专家正常进行计算
- 反向传递：专家梯度分发收集克隆的梯度并规约到原始专家，确保一致性

**ECHO** 规划器：通过 bin-packing 算法将溢出 token 与空闲容量匹配，确定克隆哪些专家到哪些 rank，使克隆开销最小化同时实现充分负载均衡。

双重收益：

1. 减少内存碎片：降低 EP ranks 间负载方差，最坏情况缓冲区大小接近实际使用，允许更小的静态缓冲区
2. 改善计算效率：均衡工作负载减少掉队者问题

### Paged Stashing (分页存储)

**ECHO** 缩小最坏情况与典型内存需求的差距；Paged Stashing 解决互补问题：在 CUDA Graph 内实现细粒度内存管理以减少内部内存碎片。

问题根源：CUDA Graphs 中的缓冲区大小需要预先确定。在简单方案中，所有层按最坏情况大小分配缓冲区，对 dropless MoE 意味着每层基于最大可能 token 数预留缓冲区，导致  $O(\text{层数} \times \text{最坏情况})$  内

存开销。

关键观察：反向计算所需激活的实际内存与为避免溢出而分配的最坏情况内存之间通常有超过一个数量级的差距。

解决方案：解耦这两个内存缓冲区：

- 1. 单个 **tmp** 缓冲区（最坏情况大小）：跨所有层共享用于计算
- 2. 分页存储缓冲区（**Stashing Buffer**）：只存储每层使用的实际 token

当一层完成前向计算后，其激活从 **tmp** 缓冲区复制到存储缓冲区（只存储实际 token 数而非最坏情况分配）。**tmp** 缓冲区随即可被后续层复用。内存消耗从  $O(\text{层数} \times \text{最坏情况})$  降至  $O(\text{最坏情况} + \text{实际总量})$ 。

存储缓冲区以 64 token/页的固定页组织（类似 OS 分页）。为最小化开销，Paged Stashing 通过专用 CUDA 流将内存传输与计算重叠：在当前计算执行期间预取下一反向层的激活。

三技术组合效果：

- 设备发起内核消除主机-设备同步，使所有 MoE 操作可被 CUDA Graphs 捕获
- ECHO 均衡专家工作负载，缩小最坏情况与实际缓冲区需求的差距
- Paged Stashing 将内存从  $O(\text{层} \times \text{最坏情况})$  降至  $O(\text{最坏情况} + \text{实际总量})$

#### 4.4 总结：突破三堵墙

| 优化方向  | 关键技术                                     | 效果                                |
|-------|------------------------------------------|-----------------------------------|
| 内存墙   | 内存高效排列、FP8/FP4 激活、细粒度重计算、卸载、精度感知优化器、FSDP | DeepSeek-V3 每 GPU 从 199.5 GB 降至可行 |
| 通信墙   | DeepEP/HybridEP 优化分发器、FWD-BWD 重叠         | all-to-all 占比从 60% 降至 <10%        |
| 计算效率墙 | 分组 GEMM、算子融合、CUDA Graphs、同步无关 MoE        | 将计算低效从主要瓶颈转变为可管理约束                |

这些优化不是独立的，而是一个有机系统：解决一堵墙的方案使其他墙的解决成为可能或得到增强。低精度训练清楚地展示了这一点——它减少内存（激活减半）和改善计算效率（更快的 Tensor Core 操作），同时需要仔细集成以避免引入新瓶颈（量化内核开销）。